0.1. Navigation & FS Basics

- `pwd`: print the current directory.
- `cd dir`, `cd ..`, `cd`, `cd ~`: change directory to *dir*, go up one level, go to home, or return to the previous directory.
- `ls [-l -a -h] path`: list files in `path`; use `-a` to include hidden files and `-h` to show human-readable sizes.
- `mkdir [-p] dir`: create directory `dir`, with `-p` making parent folders as needed; `rmdir dir`: remove the empty directory `dir`.
- `cp [-r -u] src dst`: copy files or folders from `src` to `dst`; use `-r` to copy recursively and `-u` to update only newer files; `mv src dst`: move or rename `src` to `dst`.
- `find path -name '*.py'`: search recursively under `path` for files ending in `.py`.
- Special: `.`: current directory; `..`: parent directory;`~`: user's home directory; globbing patterns: `*` matches zero or more chars; `?` matches exactly one char; `[ ]` matches any one of the enclosed chars.
- Typical system utilities (e.g., `ls`, `wc`, `uniq`) live in `/bin` or `/usr/bin`; user-installed scripts often go in `/bin`.

0.2. Environment & Shell Expansions

- Show or set environment variables: `echo $VAR` to view a variable; `export VAR=value` to set it. For example, `PYTHONPATH` defines where Python looks for modules.
- Arithmetic and command substitution: use `echo $((a+1))` (e.g., `echo $((5*3))`) to evaluate expressions, and `$(cmd)` (e.g., `files=$(ls *.py)`) to capture command output into a variable.
- Modify your `PATH` so the shell finds programs in custom locations, for example `export PATH= /bin:$PATH` makes bash search `/bin` before other directories.

0.3. Viewing & Comparing Files

- `cat file`: display the entire contents of `file`; `more file`: display `file` one screen at a time; `less file`: display `file` interactively with backward/forward navigation.
- `head [-n N] file`: output the first N lines of `file`; `tail [-n N] file`: output the last N lines of `file`; `tail -f file`: follow new lines as `file` grows.
- `wc [-l -w -m -c] file`: count lines, words, characters, or bytes in `file`.
- `diff [-u] f1 f2`: compare `f1` and `f2` and show a unified diff (e.g., `3c3` indicates a change at line 3; `< old` versus `> new`).

0.4. I/O Redirection & Pipes

`> f`	stdout→overwrite	`>> f`	stdout→append
`2> f`	stderr→overwrite	`2>> f`	stderr→append
`&> f`	both stdout & stderr→file	`2>&1`	stderr→stdout
`|`	pipe stdout→stdin	`<(cmd)`	process substitution (input)
`>(cmd)`	process substitution	`$(cmd)`	command substitution

(Process substitution `<(cmd)` and `>(cmd)` let you treat a command's output or input as a file.)

0.5. Text Processing One-liners

- `grep [-R -n -i -v -c -E] "pattern$" .` – search files (recursively with `-R`/`-r`) for lines matching `pattern$`, showing line numbers (`-n`), ignoring case (`-i`), counting matches (`-c`), inverting match (`-v`), or using extended regex (`-E`).
- `grep "regex" file > out` – filter lines matching "regex" in `file` and save them to `out`.
- `tr 'a-z' 'A-Z'; tr ',' '\t'; tr -s '[:blank:]' '\n'` – translate characters: convert lowercase to uppercase, replace commas with tabs, or squeeze consecutive blanks into newlines.
- `sort -t, -k2,2n file` – sort `file` by its second column numerically using comma as delimiter; `sort -r` – reverse the sort order.
- `uniq -c` – count identical adjacent lines in (sorted) input and prefix each with its occurrence count.
- `sort file | uniq -c | sort -nr` – produce a descending frequency count of lines in `file` by sorting, counting with `uniq -c`, and resorting numerically in reverse.
- `tr '\t' ',' < input.txt > new.txt` – convert all tabs in `input.txt` to commas and write results to `new.txt`.

0.6. Permissions & Jobs

- `chmod u+x script.sh` – add user-execute permission so you can run `script.sh`.
- `jobs`; `fg %job`; `bg %job`; `Ctrl-C` – list background jobs, bring a job to the foreground, send it to the background, or terminate the current foreground job.
- `man cmd` – show the manual page for `cmd`, using `[ ]` and `|` in the syntax examples.

1. Shell Scripting Patterns

- Shebang: `#!/usr/bin/env bash`; `set -euo pipefail` stops the script if any command fails or an undefined variable is used.
- Arguments: `$0`, `$1`, `$@`, `$#`, `$?` where `$1` is the first argument passed to the script, `$#` is the number of arguments, and `$?` is the exit code of the last command.
- Tests: use `[[ -f f ]]` to check if file *f* exists and `[[ $x -gt 0 ]]` to test if the numeric variable `$x` is greater than zero.
- **If statement**: see Code Appendix **??** for how to write standard conditional logic in a script.
- **Loops**: see Code Appendix **??**.
- **Timing**: see Code Appendix **??**.

2. Python Scripting & File I/O

2.1. Script Skeleton

- **Imports**: `import sys, os, getopt, time`
- **main function**: see Code Appendix **??** (Section **??**).
- **CLI parsing**: see Code Appendix **??** (Section **??**).

2.2. File I/O & Streams

- Open file: see Code Appendix **??**.
- Modes: `'r','w','a','rb','wb'`. (`'r'` for read, `'w'` for write, `'a'` for append, `'b'` for binary)
- Streams: `sys.stdin`, `sys.stdout`, `sys.stderr`.
- Read: `f.readline()`, `f.readlines()`, `for l in f:`. (ways to get data from a file)
- Write: `f.write("txt\n")`; `f.flush()`. (send data to a file and force it to appear immediately)

2.3. Timing & Profiling

- Basic timing in Python: see Code Appendix **??**.
- Alternatively, use `cProfile` or the external `/usr/bin/time`.

3. Abstract Data Types (ADTs)

3.1. ADT Fundamentals

- **ADT**: defines what a data structure can do without saying how it does it

3.2. Stack (LIFO)

- **Definition**: Last-In, First-Out: the last item added is the first one removed.
- **Ops**: *push(i)*, *pop()*, *peek()*, *isEmpty()*, *size()*: add to top, remove from top, look at top, check if empty, check size.
- **Implementation**:
  - *Array*: $\mathcal{O}(1)$ amortized *push*, $\mathcal{O}(1)$ for *pop*/*peek*/*isEmpty*/*size* (uses a dynamic array so pushing is usually constant time).
  - *Singly Linked List (SLL)*: $\mathcal{O}(1)$ *push*/*pop*/*peek*/*isEmpty*; $\mathcal{O}(1)/\mathcal{O}(n)$ *size* (nodes let you add/remove from top in constant time; size requires traversal if not tracked).
- **Key point**: Delimiter matching via *push*/*pop*.

3.3. Singly Linked List

- **Struct**: Node with *v,n*; head=*h*, tail=*t*: each node stores a value and a pointer to the next node, and you keep track of the first (head) and last (tail) nodes.
- **Node class**: see Code Appendix **??** for an example of defining a node in Python.
- **Core operations**:
  - *insH(v)* ($\mathcal{O}(1)$): see Code Appendix **??**—adds a new node at the front in constant time.
  - *remH()* ($\mathcal{O}(1)$): see Code Appendix **??**—removes the front node in constant time.
  - *insT(v)* ($\mathcal{O}(1)$ if tail tracked): see Code Appendix **??**—adds a new node at the end quickly when you store the tail pointer.
  - *remT()* ($\mathcal{O}(n)$): see Code Appendix **??**—removes the last node but requires scanning to find the node before it.
  - *size(), isEmpty()*: $\mathcal{O}(1)$ if you maintain a counter, otherwise $\mathcal{O}(n)$ because you must traverse all nodes to count.
- **Edge cases**: empty list, singleton list—handle scenarios when there are no nodes or only one node.
- **No random access**: nodes are not contiguous, so to reach index i you must traverse from head in $\mathcal{O}(i)$.
- **Remove tail**: $\mathcal{O}(n)$ because you must scan to find the second-to-last node before deleting the tail.
- **Key point**: identify which operations run in $\mathcal{O}(1)$ versus $\mathcal{O}(n)$.

3.4. Queue (FIFO)

- **Definition**: First-In, First-Out – the earliest enqueued element is the first dequeued.
- **Ops**: *enqueue(i)*, *dequeue()*, *front()*, *isEmpty()*, *size()*.
- **Implementations**:
  - *Circular array* — see Code **??**; all ops amortised $\mathcal{O}(1)$.
  - *Singly linked list* with head + tail pointers — strict $\mathcal{O}(1)$ *enqueue*/*dequeue*.
- **Bounded queues**: capacity fixed; *enqueue* raises overflow when full.
- **Iterator**: returns elements head→tail.
- **Trap**: a standard queue does *not* support *front(i)* for deep random access.

3.5. Deque (Double-Ended Queue)

- **Definition**: supports constant-time insert and delete at both ends—add or remove nodes from either front or back in $\mathcal{O}(1)$.
- **Implementation**: use a doubly linked list (DLL) where each node has *p,n* pointers and you track head and tail to allow operations at both ends.
- **Ops**: *addF(i)*, *addL(i)*, *remF()*, *remL()*, *isEmpty()*, *size()*—all $\mathcal{O}(1)$, since adding or removing at either end is constant time and checking emptiness or size is constant.
- **Key point**: because each node has a pointer to its predecessor (*p*), you can remove from the tail in $\mathcal{O}(1)$.
- **Use Deque as Stack**: *push*$(i) \rightarrow$ *addF*(i), *pop*$() \rightarrow$ *remF*$()$, treating the front as the top of the stack.

3.6. Priority Queue

- **Definition**: Elements have priority; *removeMin()* removes the lowest-priority item first.
- **Ops**: *ins(i,p)*, *min()*, *remMin()*—insert with priority, view smallest, remove smallest.
- **Implementation & Comparison**:
  - Unordered linked list: *ins* $\mathcal{O}(1)$; *min*/*remMin* $\mathcal{O}(n)$ because you must scan the list.
  - Ordered linked list: *ins* $\mathcal{O}(n)$; *min*/*remMin* $\mathcal{O}(1)$ since the first node is always the minimum.
  - Bounded array of linked lists: all operations $\mathcal{O}(1)$ if priorities are in a limited range, using buckets for constant-time access.
  - Min-heap: *ins*/*remMin* $\mathcal{O}(\log n)$; *min* $\mathcal{O}(1)$ using a binary tree stored in an array.
- **FIFO tie-break**: store each element as (p, i) where i is the insertion counter so items with equal priority come out in insertion order; see Code Appendix **??**.
- **Key point**: Trace the steps on a heap array or linked-list sequence when inserting or removing.

3.7. Map (Hash Map)

- **Definition**: Stores unique (key, value) pairs for fast retrieval by key.
- **Ops**: *contains(k)*, *get(k)*, *put(k,v)*, *rem(k)*, *size()*, *isEmpty()*, *keyArr()*, *toArr()*, *itCreate()*: check if a key exists, retrieve its value, add or remove a pair, get size, check if empty, get all keys, get all pairs, or create an iterator.
- **Implementation**: Array of buckets (linked lists) with chaining for collisions—each index points to a linked list of (key,value) nodes or is null if empty.
- **Hash function**: see Code Appendix **??** for an example of computing a string's hash code.
- **Time complexity**:
  - **Load factor (LF)**: $LF = M/N$, where M is the number of entries and N is the number of buckets; expected entries per bucket = *LF*.
  - *put*, *get*, *contains*, *rem*: $\mathcal{O}(1)$ average, $\mathcal{O}(n)$ worst (usually constant time unless many items collide).
  - *keyArr*, *toArr*: $\mathcal{O}(n)$ to copy all keys or pairs into an array.
  - *itCreate*: $\mathcal{O}(1)$ to start; next() amortized $\mathcal{O}(1)$.
- **Iterator sketch**: see Code Appendix **??** for an example of looping through all keys.
- **Key point**: Illustrate resize, hash index computation, collision handling, and bucket traversal in a hash table.
- **Applications**: Word frequency counting, magazine-note problems, anagram grouping.
- **Good hash function**:
  - Deterministic: the same input always yields the same hash.
  - Uniformly distributes keys across buckets to minimize collisions.
  - Fast to compute (e.g., polynomial hashing modulo table size).

  Bucket-array organisation. Each slot stores a pointer to the **head node** of a singly linked list of (*key*, *value*) pairs or `None` if empty); data never sit directly in the array itself (Code **??**).

  Load-factor recap. With M entries and N buckets, LF $= M/N$. Rehash when $M > \alpha N$ to keep expected chain length $< \alpha$ and retain average $\mathcal{O}(1)$ performance.

  Hash-function rules. A hash must be *deterministic*, uniformly distributed, and fast. Using `random(value)` breaks determinism – identical keys land in different buckets, destroying look-ups (MC #9 rationale).

  toArray() vs itCreate(). Call *toArray()* when you need indexed/random access to every pair ($\mathcal{O}(n)$ time & space). Use *itCreate()* for one-pass scans with $\mathcal{O}(1)$ extra storage.

3.8. Heap (Min-Heap)

- **Definition**: Complete binary tree with min-heap property stored as an array, where each parent is its children.
- **Ops**:
  - `siftup(i)` ($\mathcal{O}(\log n)$): see Code Appendix **??**—move a new or adjusted item up until heap order is restored.
  - `siftdown(i)` ($\mathcal{O}(\log n)$): see Code Appendix **??**—move a node down to reestablish the heap property.
  - *ins(i)*: append element to the end of the array and run `siftup` to fix the heap.
  - *min()*: return `heap[0]` in $\mathcal{O}(1)$ since the smallest item is always at index 0.
  - *remMin()*: swap the root with the last element, pop the last element, then run `siftdown(0)` to restore heap order.
- Build-heap: in $\mathcal{O}(n)$ by running `siftdown` from $\lfloor n/2 \rfloor$ down to 0 to transform an unordered array into a valid heap.
- **Key point**: show build steps, trace `siftup` calls, and prove why build-heap runs in $\mathcal{O}(n)$.

  Min-Heap vs Max-Heap
- **Min-Heap**: parent $\leq$ children; *remMin()* extracts the smallest key.
- **Max-Heap**: parent $\geq$ children; *remMax()* extracts the largest key.
- **Complexities**: both flavours keep *insert* and removal in $\mathcal{O}(\log n)$.
- **Heapsort rationale**: building a *max*-heap lets repeated *remMax()* place each maximum at the *end* of the array, producing an ascending in-place sort. Doing the same with a min-heap yields a descending order (reason for MC #19).
- **FIFO tie-break** in a min-heap priority queue: store a monotonically increasing sequence number with each key (Code **??**); equal priorities then dequeue in insertion order (addresses MC #17).

3.9. Cross-Cutting Details

- **NumPy arrays**: dynamically resize by doubling capacity, so appends are $\mathcal{O}(1)$ amortized with occasional $\mathcal{O}(n)$ when resizing.
- **ArrayQueue enqueue cost**: similar to dynamic arrays—if the backing array is full, it reallocates and copies all n elements, making that operation $\mathcal{O}(n)$, but amortized $\mathcal{O}(1)$.
- **Custom exceptions**: define your own error types with, e.g., `class E(Exception): pass`.
- **Iterator support**: *itCreate()* returns an iterator implementing `___next___`, which raises `StopIteration` when there are no more elements.
- `ADTtypemap.py`: provides type-conversion utilities between Python primitives and ADT types.

Topic	Key Correction (MC #)
Queue facts	No deep *front(i)* access; iterator = head→tail (4)
toArray vs iterator	Use toArray for random access, iterator for scan (5)
Stable sort	Preserve input order of equal keys (8)
Hash bucket array	Stores list-head pointer per bucket (16)
FIFO in min-heap PQ	Add sequence number to key tuple (17)
Max-heap in heapsort	Removes max to build ascending array (19)

4. Complexity & Sorting

4.1. Big-O Classes

- $\mathcal{O}(1)$, $\mathcal{O}(\log n)$, $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, $\mathcal{O}(n^2)$: common categories for how runtime grows with input size.
- **Worst, Average, Amortized**:
  - Worst: the maximum time an operation takes for input size n.
  - Average: the expected time over a distribution of inputs.
  - Amortized: the average time per operation over a sequence (e.g., appending to a dynamic array has occasional slow resizes but is $\mathcal{O}(1)$ on average).
- Examples: appending to a dynamic array is amortized $\mathcal{O}(1)$, and binary search is $\mathcal{O}(\log n)$.
- **Python XOR**: `b1 ^ b2` returns True exactly when one of `b1` or `b2` is True (exclusive-or).

Stable sort. An algorithm is **stable** iff equal-key elements preserve their original left-to-right order: $O_1 \prec O_2 \wedge key(O_1) = key(O_2) \Rightarrow O_1 \prec O_2$ in output. Determinism alone does *not* imply stability.

4.2. Sorting Comparison

Sort	Avg./Worst	Stable?	In-place?
Insertion	$\mathcal{O}(n^2)/\mathcal{O}(n^2)$	Yes	Yes
Selection	$\mathcal{O}(n^2)/\mathcal{O}(n^2)$	No	Yes
Bubble (exchange)	$\mathcal{O}(n^2)/\mathcal{O}(n^2)$	Yes	Yes
Shell	$\mathcal{O}(n^{1.4})$– $\mathcal{O}(n^{5/3})$	No	Yes
Heap	$\mathcal{O}(n \log n)/\mathcal{O}(n \log n)$	No[1]	Yes
Quick	$\mathcal{O}(n \log n)/\mathcal{O}(n^2)$	No[2]	Yes
Merge	$\mathcal{O}(n \log n)/\mathcal{O}(n \log n)$	Yes	No

- **Stable sorts**: Insertion, Bubble, Merge. (if two items compare equal, they stay in the same order as in the input)
- **Unstable sorts**: Selection, Quick, Heap, Shell. (equal items may be reordered)
- **QuickSort worst-case:** when pivot choice yields extremely unbalanced partitions (e.g. always picking first or last element on already sorted data). (leads to $\mathcal{O}(n^2)$ behavior when cuts are uneven)

5. Code Appendix

5.1. A.1 If Statement

```
if [[ $x -gt 0 ]]; then echo "pos"
elif [[ $x -lt 0 ]]; then echo "neg"
else echo "zero"; fi
```

(choose a branch based on whether x is positive, negative, or zero)

5.2. A.2 Script Skeleton

```
def main():
    # parse args + logic
if __name__=="__main__":
    main()
```

(standard way to start a Python script so main() runs when the file is executed)

5.3. A.3 CLI Parsing

```
import getopt, sys
try:
    opts,args=getopt.getopt(sys.argv[1:],"hi:o:")
except getopt.GetoptError as e:
    print(e); sys.exit(2)
for o,a in opts:
    if o=="-h": pass
    elif o=="-i": inf=a
# args
```

(parse command-line options like -h or -i <file> and handle errors)

5.4. A.4 Node Class

```
class N:
    def __init__(self,v):
        self.v=v; self.n=None
```

(a simple node for a linked list, with a value and a pointer to the next node)

5.5. A.5 insH

```
x.n=self.h; self.h=x
if self.t is None: self.t=x
```

(insert a node at the head; if the list was empty, also set tail)

5.6. A.6 remH

```
if self.h is None: raise E
r=self.h.v; self.h=self.h.n
if self.h is None: self.t=None
return r
```

(remove and return the head node; if list becomes empty, clear tail)

5.7. A.7 insT

```
x=N(v)
if self.t is None: self.h=x; self.t=x
else: self.t.n=x; self.t=x
```

(insert a node at the tail; if list was empty, set head and tail)

5.8. A.8 remT

```
if self.h is None: raise E
if self.h==self.t:
    r=self.h.v; self.h=self.t=None; return r
p=self.h
while p.n!=self.t: p=p.n
r=self.t.v; p.n=None; self.t=p; return r
```

(remove and return the tail node; find the node before tail to update pointers)

5.9. A.9 realComp

```
def cmp(a,b):
    # a,b=(p,s,i)
    if a.p<b.p: return True
    if a.p==b.p: return a.s<b.s
    return False
```

(compare priority and insertion order so ties go in FIFO order)

5.10. A.10 strHash

```
def ph(s,ts):
    h=0
    for c in s: h=(h*31+ord(c))%ts
    return h
```

(compute a simple hash for a string by combining character codes)

5.11. A.11 MapIter

```
class MI:
    def __init__(self,b):
        self.b=b; self.i=0
        self.n=self.adv(0)
    def __next__(self):
        if self.n is None: raise StopIteration
        k=self.n.k
        self.n=self.n.n or self.adv(self.i+1)
        return k
```

(iterate through all keys in a hash map, advancing through buckets)

5.12. A.12 siftup

```
while i>0:
    p=(i-1)//2
    if heap[i]<heap[p]:
        swap(heap,i,p); i=p
    else: break
```

(move an element at index i upward until the heap property is restored)

5.13. A.13 siftdown

```
n=len(heap)
while True:
    l=2*i+1; r=2*i+2; m=i
    if l<n and heap[l]<heap[m]: m=l
    if r<n and heap[r]<heap[m]: m=r
    if m!=i: swap(heap,i,m); i=m
    else: break
```

(move an element at index i downward until the heap property is restored)

5.14. A.14 Shell Loops

```
for f in *.py; do echo "$f"; done
while [[ cond ]]; do # cmds; done
```

(loop over every .py file or repeat commands while condition is true)

5.15. A.15 Shell Timing

```
/usr/bin/time -f "%E elap, %MKB mem" prog
```

(run prog and report elapsed time and peak memory use)

5.16. A.16 Open File (Python)

```
with open(f, 'r') as f:
    for l in f: # process l
```

(open a file for reading and loop over each line)

5.17. A.17 Python Timing

```
import time
t0 = time.perf_counter()
# run code
dt = time.perf_counter() - t0
print(f"{dt:.4f}s")
```

(measure elapsed time in seconds for a block of code)

5.18. A.18 ArrayQueue Skeleton

```
class ArrayQueue:
    def __init__(self, cap:int=8):
        self._a   = [None]*cap   # circular buffer
        self._f   = 0            # index of front item
        self._len = 0

    def _resize(self, ncap:int) -> None:
        b = [None]*ncap
        for i in range(self._len):
            b[i] = self._a[(self._f+i) % len(self._a)]
        self._a, self._f = b, 0

    # O(1) amortised
    def enqueue(self, x):
        if self._len == len(self._a):
            self._resize(2*len(self._a))
        back = (self._f + self._len) % len(self._a)
        self._a[back] = x
        self._len += 1

    # raises EmptyError if empty
    def dequeue(self):
        if self._len == 0: raise EmptyError
        x = self._a[self._f]; self._a[self._f] = None
        self._f = (self._f + 1) % len(self._a)
        self._len -= 1
        return x
```

(*FIFO queue with circular array; all methods $\mathcal{O}(1)$ amortised.*)

5.19. A.19 reverseStack() via Queue

```
def reverseStack(st:"ArrayStack") -> None:
    q = ArrayQueue(st.size())
    while not st.isEmpty():     # pop → enqueue
        q.enqueue(st.pop())
    while not q.isEmpty():      # dequeue → push
        st.push(q.dequeue())
```

(*Two linear sweeps give in-place stack reversal; matches practice-test code.*)

5.20. A.20 Stable Merge Sort (Python)

```
def mergesort(a:list) -> list:
    n = len(a)
    if n <= 1: return a
    mid = n//2
    L = mergesort(a[:mid])
    R = mergesort(a[mid:])
    i=j=0; out=[]
    while i<len(L) and j<len(R):
        if L[i] <= R[j]:        # '<='   stability
            out.append(L[i]); i += 1
        else:
            out.append(R[j]); j += 1
    out.extend(L[i:]); out.extend(R[j:])
    return out
```

(*Divide-and-conquer; $\Theta(n \log n)$, stable.*)

5.21. A.21 FIFO Tie-break in Min-Heap PQ

```
class _PQEntry:
    __slots__ = ("p","seq","val")
    def __init__(self, p, seq, val):
        self.p, self.seq, self.val = p, seq, val
    def __lt__(self, other):            # min-heap order
        return (self.p, self.seq) < (other.p, other.seq)

class HeapPrioQueue:
    def __init__(self):
        self._h = []; self._seq = 0
    def insert(self, priority, value):
        heapq.heappush(self._h,
                       _PQEntry(priority, self._seq, value))
        self._seq += 1                  # increments each insert
```

(*Pair priority with monotone sequence equal-priority items emerge FIFO.*)

5.22. A.22 Quicksort Partition (1st-pivot Worst Case)

```
def quicksort(a:list, lo:int=0, hi:int=None) -> None:
    if hi is None: hi = len(a)-1
    if lo >= hi: return
    p = a[lo]                   # BAD on presorted arrays
    i, j = lo+1, hi
    while True:
        while i<=j and a[i] <= p: i += 1
        while a[j] > p:       j -= 1
        if i > j: break
        a[i],a[j] = a[j],a[i]
    a[lo],a[j] = a[j],a[lo]   # pivot   split point
    quicksort(a, lo, j-1); quicksort(a, j+1, hi)
```

(*Choosing first element as pivot on sorted input $O(n^2)$ partitions.*)

5.23. A.23 Bucket-Array Layout for HashMap

```
# organisational array 'buckets'   chaining
# idx → head of singly-linked list or None
buckets : list[Node | None] = [None] * N
```

(*Each bucket holds a listhead; nodes store (key,value,next).*)

5.24. A.24 Polynomial String Hash

```
def polyHash(s:str, mod:int) -> int:
    h = 0
    for ch in s:
        h = (h*37 + ord(ch)) % mod   # 37   small prime base
    return h
```

(*Deterministic, uniform, $O(|s|)$; adjust base/mod for table size.*)

5.25. A.25 printPrioQueue() using toArray()

```
def printPrioQueue(pq, fd) -> None:
    for _,s in pq.toArray():     # array of (prio,str) tuples
        fd.write(f"{s}\n")
```

(*Straight from practice-test spec; array allows indexed access, $O(n)$.*)

5.26. A.26 selSort

```
def sort(a, cmp):
    i = 0
    while i < len(a) - 1:
        k = i
        j = i + 1
        while j < len(a):
            if cmp(a[j], a[k]) < 0:
                k = j
            j += 1
        if k != i:
            x = a[i]
            a[i] = a[k]
            a[k] = x
        i += 1
```

(selection sort; not stable; $\Theta(n^2)$)

5.27. A.27 heapSort

```
def ind(val: int) -> int:
    return val - 1

def sift(b, L, R, cmp):
    """Establish max-heap on b[L]..b[R]; pre: heap(L+1,R)"""
    i = L
    j = 2 * L
    x = b[ind(L)]
    if j < R and cmp(b[ind(j)], b[ind(j+1)]) < 0:
        j += 1
    while j <= R and cmp(x, b[ind(j)]) < 0:
        b[ind(i)] = b[ind(j)]
        b[ind(j)] = x
        i = j
        j *= 2
        if j < R and cmp(b[ind(j)], b[ind(j+1)]) < 0:
            j += 1

def sort(a, cmp):
    R = len(a)
    L = R // 2 + 1
    while L > 1:
        L -= 1
        sift(a, L, R, cmp)        # build max-heap
    while R > 1:
        x = a[ind(1)]              # swap root with last
        a[ind(1)] = a[ind(R)]
        a[ind(R)] = x
        R -= 1
        sift(a, 1, R, cmp)        # restore heap
```

(heap sort; not stable; $\Theta(n \log n)$)

5.28. A.28 insSort

```
def sort(a, cmp):
    i = 1
    while i < len(a):
        x = a[i]
        j = i - 1
        while j >= 0 and cmp(a[j], x) > 0:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = x
        i += 1
```

(insertion sort; stable; $\Theta(n^2)$)

5.29. A.29 exchSort

```
def sort(a, cmp):
    nexchange = 1
    i = 1
    while i < len(a) and nexchange > 0:
        j = len(a) - 1
        nexchange = 0
        while j >= i:
            if cmp(a[j-1], a[j]) > 0:
                x = a[j-1]
                a[j-1] = a[j]
                a[j] = x
                nexchange += 1
            j -= 1
        i += 1
```

(exchange/bubble sort (improved); stable; $\Theta(n^2)$, often faster when nearly sorted)

5.30. A.30 shellSort

```
import numpy as np

def logb2(size: int) -> int:            # log2(size) + 1
    power = 1
    mask = 1
    while size > mask:
        mask <<= 1
        power += 1
    return power

def sort(a, cmp):
    t = logb2(len(a)) - 1               # number of strides
    h = np.empty(t, int)
    h[t-1] = 1
    for i in range(t-2, -1, -1):
        h[i] = 2 * h[i+1] + 1           # Pratt sequence 1,3,7
    for stride in h:
        for j in range(stride, len(a)):
            k = j - stride
            while k >= 0 and cmp(a[k+stride], a[k]) < 0:
                a[k], a[k+stride] = a[k+stride], a[k]
                k -= stride
```

(shell sort; not stable; worst-case $\Theta(n^{5/3})$, typically $\Theta(n^{1.25})$ or better)

5.31. A.31 quickSortIter

```
import numpy as np

class pStruct:                  # partition descriptor
    def __init__(self, l, r):
        self.left = l
        self.right = r

def logb2(size) -> int:
    power = 1
    mask = 1
    while size > mask:
        mask <<= 1
        power += 1
    return power

def sort(a, cmp):
    maxDepth = logb2(len(a))
    st = np.empty(maxDepth, dtype=object)
    st[0] = pStruct(0, len(a) - 1)
    sp = 0                                  # stack pointer
    while sp >= 0:
        L, R = st[sp].left, st[sp].right
        sp -= 1                             # pop
        while L < R:
            i, j = L, R
            x = a[(L+R)//2]                 # pivot
            while i <= j:
                while cmp(a[i], x) < 0: i += 1
                while cmp(x, a[j]) < 0: j -= 1
                if i <= j:
                    a[i], a[j] = a[j], a[i]
                    i += 1; j -= 1
            if (j-L) < (R-i):               # push larger pa
                if i < R:
                    sp += 1; st[sp] = pStruct(i, R)
                R = j
            else:
                if L < j:
                    sp += 1; st[sp] = pStruct(L, j)
                L = i
```

(iterative quick sort; not stable; average Θ(n log n), worst $\Theta(n^2)$)

5.32. A.32 mergeSort

```
import numpy as np
scratch = None                  # global work array

def isort(a, cmp):              # insertion for tiny slices
    for i in range(1, len(a)):
        x = a[i]; j = i - 1
        while j >= 0 and cmp(a[j], x) > 0:
            a[j+1] = a[j]; j -= 1
        a[j+1] = x

def merge(a, L, M, H, cmp):
    l1, l2, i = L, M+1, L
    while l1 <= M and l2 <= H:
        if cmp(a[l1], a[l2]) <= 0:
            scratch[i] = a[l1]; l1 += 1
        else:
            scratch[i] = a[l2]; l2 += 1
        i += 1
    while l1 <= M:
        scratch[i] = a[l1]; i += 1; l1 += 1
    while l2 <= H:
        scratch[i] = a[l2]; i += 1; l2 += 1
    for i in range(L, H+1):
        a[i] = scratch[i]

THRESHOLD = 7
def msort(a, low, high, cmp):
    size = high - low + 1
    if size > THRESHOLD:
        mid = (low + high) // 2
        msort(a, low, mid, cmp)
        msort(a, mid+1, high, cmp)
        merge(a, low, mid, high, cmp)
    elif size > 1:
        isort(a[low:high+1], cmp)

def sort(a, cmp):
    global scratch
    scratch = np.empty(len(a), dtype=a.dtype)
    msort(a, 0, len(a)-1, cmp)
    scratch = None
```

(merge sort; stable; Θ(n log n); switches to insertion sort for tiny slices)